

Operations Research

Convex Optimization

Prof. Dr. Martin Bichler

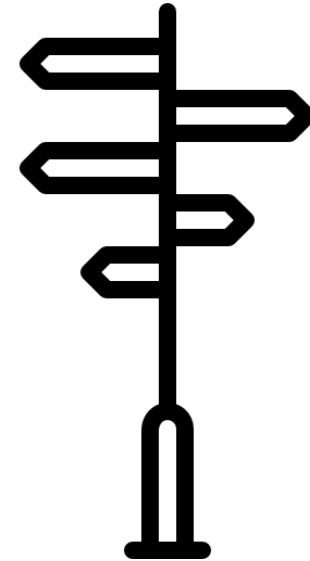
Department of Computer Science

School of Computation, Information, and Technology

Technical University of Munich

Course outline

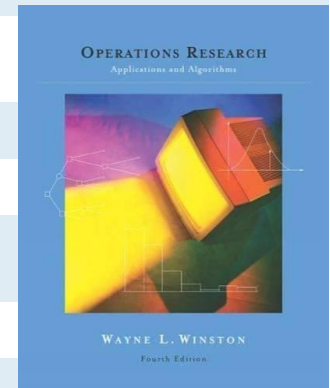
- **Linear Optimization**
 - Modeling by linear programming and graphical solution
 - Solving linear programs, convexity
 - Simplex algorithm, 2-phase simplex
 - Simplex algorithm in matrix notation, goal programming
 - Sensitivity analysis, reoptimization
 - Duality theory, zero-sum games
- **Integer Linear Optimization**
 - Modeling integer programming problems (IPs)
 - Solving IPs: branch & bound, cutting planes
 - Advanced solution methods: column generation, approximation algorithms
 - Fast solvable integer problems: total unimodularity, matroids
 - Network flow problems
 - Traveling salesperson problem
- **Nonlinear Optimization**
 - Optimization of multidimensional functions
 - Convex optimization
 - **Online convex optimization**



Operations Research: Applications and Algorithms

Wayne L. Winston, 4th Edition

Course Lecture	Winston Chapter
2 LP I: Modeling with LP	Ch. 3 – Introduction to LP
3 LP II: Solving LPs	Ch. 3–4
4 LP III: Simplex, 2-phase, Big-M	Ch. 4 – Simplex Algorithm & Goal Programming
5 LP IV: Matrix notation, goal programming	Ch. 4
6 Sensitivity analysis, reoptimization	Ch. 5 & 6
7 Duality theory, zero-sum games	Ch. 6 – Sensitivity Analysis & Duality, Ch. 14 – Game Theory
8 Modeling IPs	Ch. 9 – Integer Programming
9 Branch & bound, cutting planes	Ch. 9 – Integer Programming
10 Column generation, approximation algorithms	Column generation Approximation algorithms
11 Total unimodularity, matroids	DP for knapsack (Wikipedia) , total unimodularity , Matroids
12 Network flow problems	Ch. 7 (transportation/assignment) & Ch. 8 (network models)
13 Traveling Salesperson Problem	Ch. 9 – Integer Programming
14 Nonlinear optimization	Ch. 12 – Nonlinear Programming
15 Convex optimization	Ch. 12.6 - 12.8 – Nonlinear Programming
16 Online optimization	Online Convex Optimization



Gradient *descent* method with constraints

Pseudocode:

$k = 1$

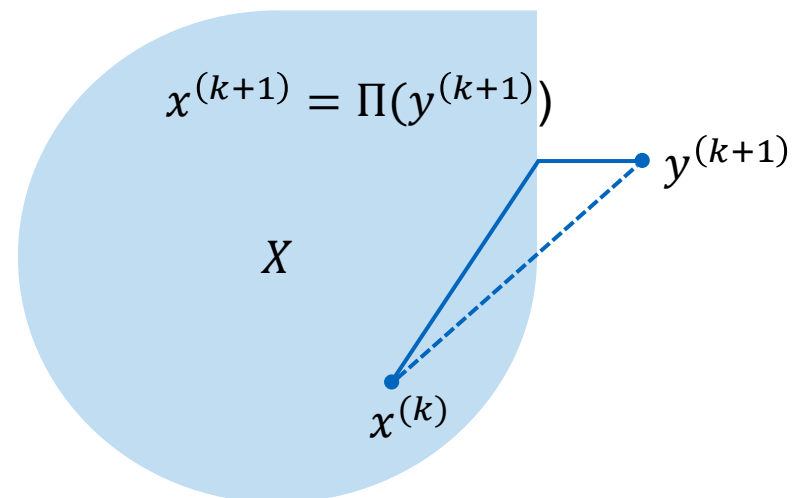
While($\|\nabla f(x^{(k)})\| \geq \varepsilon$) {

- $y^{(k+1)} = x^{(k)} - \eta \nabla f(x^{(k)})$

- $x^{(k+1)} = \Pi(y^{(k+1)}) = \arg \min_{x \in X} \|y^{(k+1)} - x\|$ //Euclidean projection

- $k++$

}



Example of projected gradient descent

A company can produce two products A and B .

x = quantity A , y = quantity B

The maximum total (normalized) quantity that can be produced is 1: $x + y \leq 1$

A is sold for price 5 €, B for 7 €.

Costs incurred:

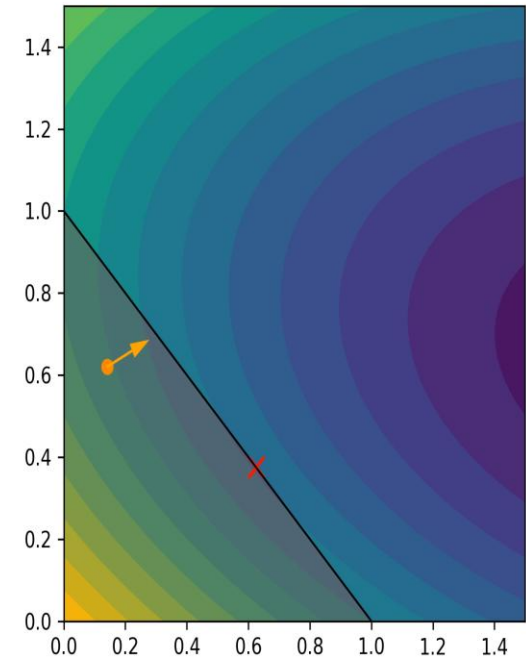
$$x^2 + xy + 4y^2 = (x, y) \begin{pmatrix} 1 & 0.5 \\ 0.5 & 4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Maximize profit ($\Leftrightarrow$ minimize loss):

$$\min (x, y) \begin{pmatrix} 1 & 0.5 \\ 0.5 & 4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} - (5x + 7y)$$

$$\text{s.t. } x + y \leq 1$$

$$x, y \geq 0$$



Convergence of the projected gradient descent method

Theorem: For step size $\eta = \frac{D}{G\sqrt{T}}$

$$f\left(\frac{1}{T}\sum_{k \leq T} x^{(k)}\right) \leq \min_{x^* \in X} f(x^*) + \frac{DG}{\sqrt{T}}$$

Where:

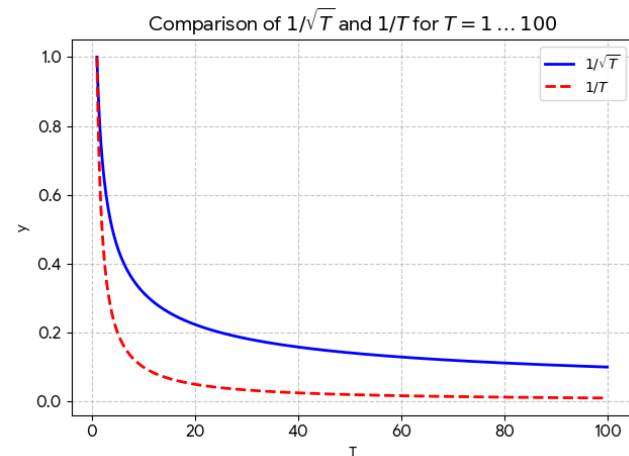
G is an upper bound for the norm of the gradient, i.e. steepness of the function:

$$\|\nabla f(x^{(k)})\| \leq G$$

D is the diameter of the polytope formed by the constraints:

$$\forall x, y \in X: \|x - y\| \leq D$$

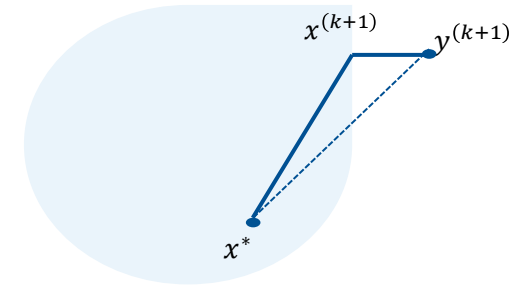
In words, the function value at the average of the iterates is at most the optimal value plus an error term. This means, the method converges at rate $O(\frac{1}{\sqrt{T}})$.



$$y^{(k+1)} = x^{(k)} - \eta \nabla f(x^{(k)})$$
$$x^{(k+1)} = \arg \min_{x \in X} \|y^{(k+1)} - x\|$$

$$y^{(k+1)} = x^{(k)} - \eta \nabla f(x^{(k)})$$

$$x^{(k+1)} = \arg \min_{x \in X} \|y^{(k+1)} - x\|$$



Proof of convergence rate

Observation 1: $\|y^{(k+1)} - x^*\|^2 = \|x^{(k)} - \eta \nabla f(x^{(k)}) - x^*\|^2$

Observation 2: $\|x^{(k+1)} - x^*\|^2 \leq \|y^{(k+1)} - x^*\|^2$

$$(a - b)^2 = a^2 - 2ab + b^2$$

Therefore: $\|x^* - x^{(k+1)}\|^2 \leq \|x^* - x^{(k)}\|^2 - 2\eta \nabla f(x^{(k)})(x^{(k)} - x^*) + \eta^2 \|\nabla f(x^{(k)})\|^2$

Proof:

Convexity: the average point is always above the average

This is lower-bounded by the gradient

$$f(x^*) \geq f(x^{(k)}) + \frac{1}{T} \sum_{k \leq T} \nabla f(x^{(k)})(x^{(k)} - x^*)$$

$$f\left(\frac{1}{T} \sum_{k \leq T} x^{(k)}\right) - f(x^*) \leq \frac{1}{T} \sum_{k \leq T} [f(x^{(k)}) - f(x^*)] \leq \frac{1}{T} \sum_{k \leq T} \nabla f(x^{(k)})(x^{(k)} - x^*)$$

$$\leq \frac{1}{T} \sum_{k \leq T} \frac{1}{2\eta} (\|x^* - x^{(k)}\|^2 - \|x^* - x^{(k+1)}\|^2) + \frac{\eta}{2} \|\nabla f(x^{(k)})\|^2$$

$$\|x - y\| \leq D$$

$$= \frac{1}{T2\eta} (\|x^* - x^{(1)}\|^2 - \underbrace{\|x^* - x^{(T+1)}\|^2}_{\leq 0}) + \frac{\eta}{2} \|\nabla f(x^{(k)})\|^2$$

$$\|\nabla f(x^{(k)})\| \leq G$$

$$\eta = \frac{D}{G\sqrt{T}}$$

$$\leq \frac{1}{T2\eta} D^2 + \frac{\eta}{2} G^2 = \frac{DG}{2\sqrt{T}} + \frac{DG}{2\sqrt{T}} \leq \frac{DG}{\sqrt{T}}$$

Today You Learn

- **Mirror descent** as generalization of gradient descent.
- **Multi-armed bandit (MAB)** problems that deal with discrete decision spaces.
- **Online convex optimization (OCO)** problems involving continuous decision spaces, but possibly changing loss functions.



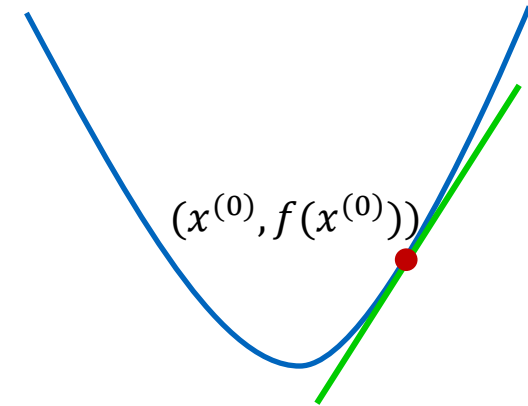
Recap: gradient *descent* method

Input: $\min f(x), s.t. x \in X$

Derivation via 1st order Taylor approximation

(i.e. via a linear function)

$$f(x) \approx f(x^{(0)}) + \langle \nabla f(x^{(0)}), x - x^{(0)} \rangle$$



This would follow the gradient arbitrarily far. However, the approximation is only good near $x^{(0)}$. Therefore, one adds a penalty term and uses the following formula, starting from $k = 0$.

$$x^{(k+1)} = \operatorname{argmin}_x f(x^{(k)}) + \langle \nabla f(x^{(k)}), x - x^{(k)} \rangle + \frac{1}{2\eta} \|x - x^{(k)}\|^2$$

$$= \operatorname{argmin}_x \langle \nabla f(x^{(k)}), x \rangle + \frac{1}{2\eta} \|x - x^{(k)}\|^2$$

$$x^{(k+1)} = \operatorname{argmin}_x \{ \eta \langle \nabla f(x^{(k)}), x \rangle + D_h(x || x^{(k)}) \}$$

Can we replace the regularizer?

Set first derivative w.r.t. x to 0:

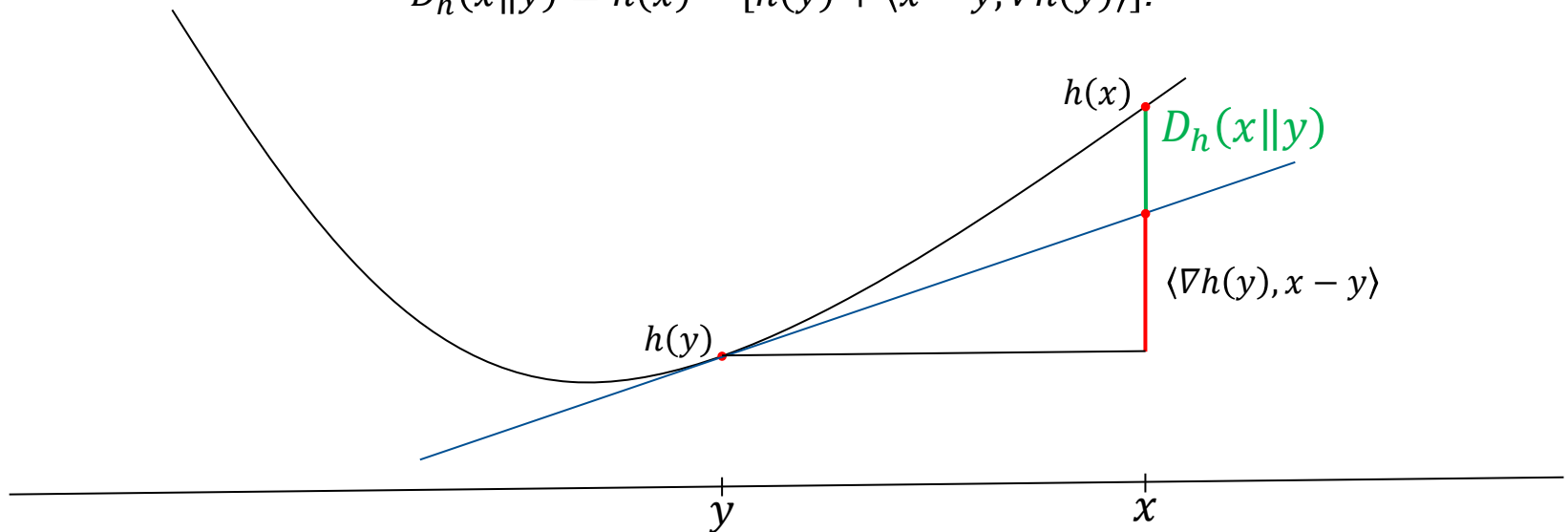
$$0 + \nabla f(x^{(k)}) + \frac{1}{\eta} (x - x^{(k)}) = 0$$

$$x^{(k+1)} = x^{(k)} - \eta \nabla f(x^{(k)})$$

Bregman divergence

For two points $x, y \in X$, differentiable function $h: X \rightarrow \mathbb{R}$ induces the Bregman divergence $D_h(x||y)$, defined as:

$$D_h(x||y) = h(x) - [h(y) + \langle x - y, \nabla h(y) \rangle].$$



Such functions should be α -strongly convex: $D_h(x||y) \geq \frac{\alpha}{2} \|x - y\|^2$

Preview of what comes next:

$h(\cdot)$ can be the **Euclidean distance** such that D_h becomes the **squared Euclidean distance**

$h(\cdot)$ can be the **negative entropy** function such that D_h becomes the **KL divergence**

Different distance functions in mirror descent

$$x^{(k+1)} = \operatorname{argmin}_x \{ \eta \langle \nabla f(x^{(k)}), x \rangle + D_h(x || x^{(k)}) \}$$

Bregman divergence

$$D_h(x || x^{(k)}) = h(x) - \left[h(x^{(k)}) + \nabla h(x^{(k)})^T (x - x^{(k)}) \right]$$

Insert

$$\operatorname{argmin}_x \left\{ \eta \langle \nabla f(x^{(k)}), x \rangle + h(x) - h(x^{(k)}) - \nabla h(x^{(k)})^T (x - x^{(k)}) \right\}$$

First-order condition w.r.t. x and set to 0

$$\eta \nabla f(x^{(k)}) + \nabla h(x) - \nabla h(x^{(k)}) = 0$$

Evaluate at point $x^{(k+1)}$

$$\eta \nabla f(x^{(k)}) + \nabla h(x^{(k+1)}) - \nabla h(x^{(k)}) = 0$$

Rearrange

$$\begin{aligned} \nabla h(x^{(k+1)}) &= \nabla h(x^{(k)}) - \eta \nabla f(x^{(k)}) \\ x^{(k+1)} &= \nabla h^{-1} \left(\nabla h(x^{(k)}) - \eta \nabla f(x^{(k)}) \right) \end{aligned}$$

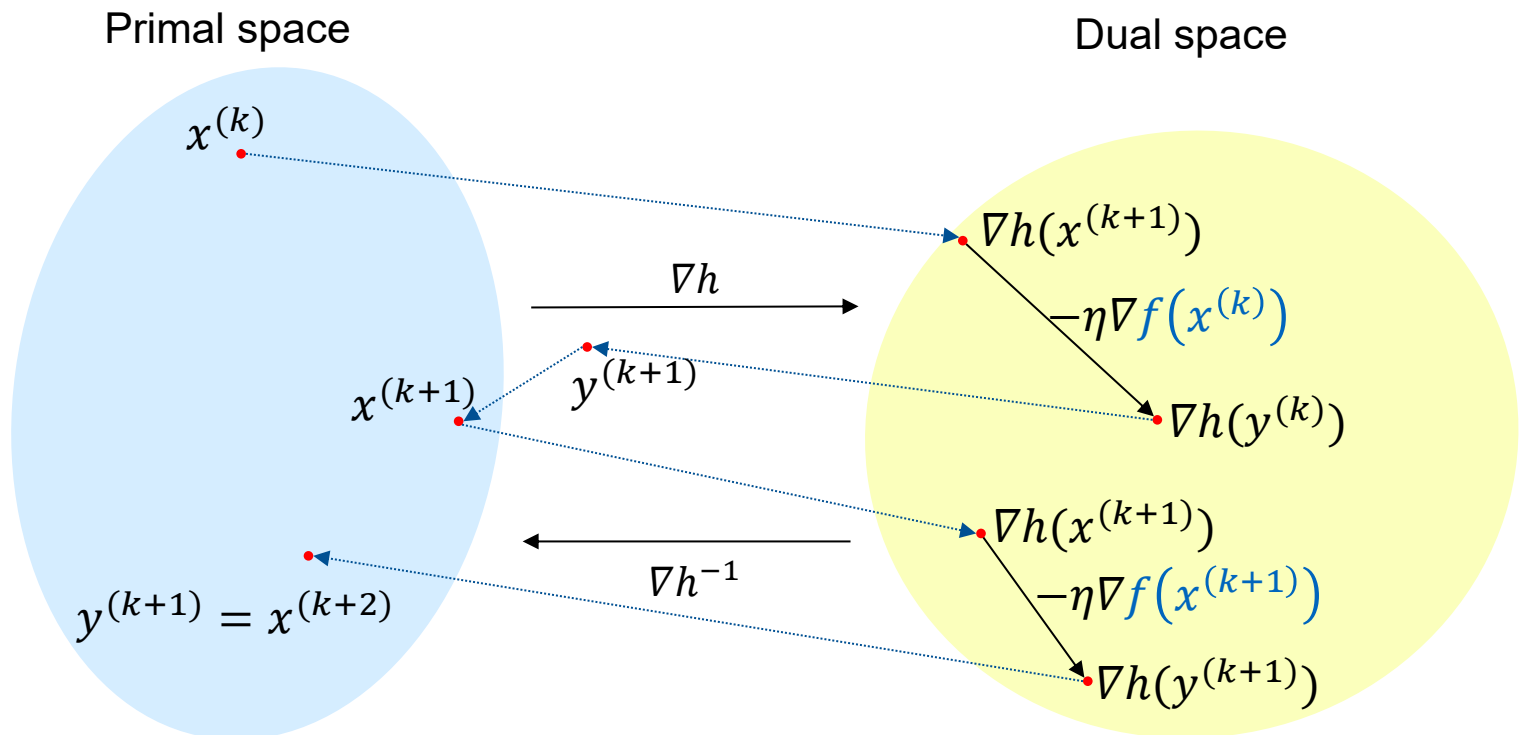
$h(x)$ is called „mirror map“

Mirror descent (in general)

Gradient steps are performed in the dual space:

$$x^{(k+1)} = \nabla h^{-1} \left(\nabla h(x^{(k)}) - \eta \nabla f(x^{(k)}) \right)$$

Sometimes $y^{(k)}$ must be projected onto the feasible primal space using Bregman divergence
 $x^{(k+1)} = \arg \min_x D_h(x || y^{(k+1)})$.



Different distance functions in mirror descent

$$x^{(k+1)} = \nabla h^{-1} \left(\nabla h(x^{(k)}) - \eta \nabla f(x^{(k)}) \right)$$

Gradient descent as a form of mirror descent:

Using **Euclidean distance** $h(x) = \frac{1}{2} \|x\|^2 = \frac{1}{2} x^T x = \frac{1}{2} (x_1^2 + x_2^2 + \dots + x_n^2)$

Bregman divergence: $D_h(x||x^{(k)}) = h(x) - \left[h(x^{(k)}) + \nabla h(x^{(k)})^T (x - x^{(k)}) \right]$

Observe: $\nabla h(x) = [x_1, \dots, x_n] = \nabla h^{-1}(x) = x$

$$D_h(x||x^{(k)}) = h(x) - \left[h(x^{(k)}) + \nabla h(x^{(k)})^T (x - x^{(k)}) \right]$$

$$D_h(x||x^{(k)}) = \frac{1}{2} x^T x - \left[\frac{1}{2} x^{(k)T} x^{(k)} + x^{(k)T} (x - x^{(k)}) \right] = \frac{1}{2} \|x - x^{(k)}\|^2,$$

the **squared Euclidean distance**

So, again we get $x^{(k+1)} = \operatorname{argmin}_x f(x^{(k)}) + \langle \nabla f(x^{(k)}), x - x^{(k)} \rangle + \frac{1}{2\eta} \|x - x^{(k)}\|^2$

Mirror descent updates become

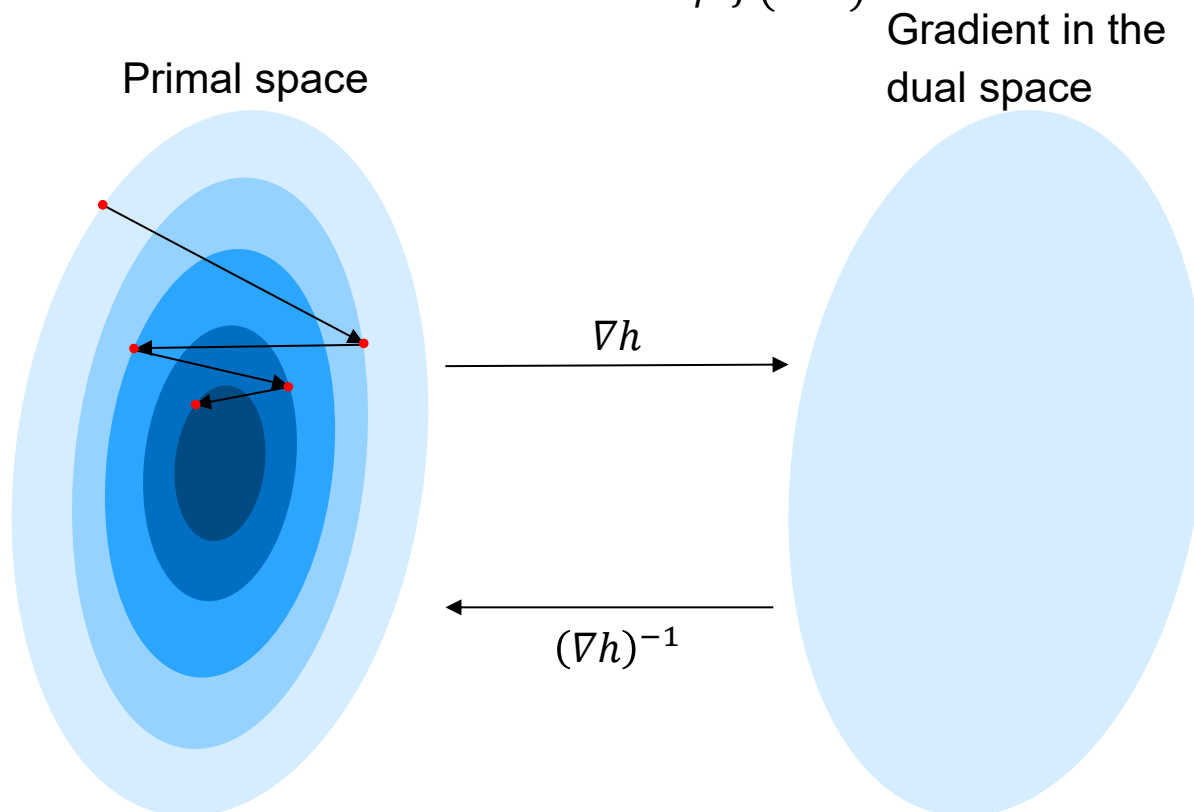
$$x^{(k+1)} = x^{(k)} - \eta \nabla f(x^{(k)})$$

This is gradient descent as a version of mirror descent.

Gradient descent

Gradients live in a dual space. This space is self-dual ($\nabla h(x) = [x_1, \dots, x_n] = \nabla h^{-1}(x) = x$) and we do not need to consider it further. With other mirror maps, this is not the case.

$$x^{(k+1)} = x^{(k)} - \eta \nabla f(x^{(k)})$$



Distance distance functions in mirror descent

$$x^{(k+1)} = \nabla h^{-1} \left(\nabla h(x^{(k)}) - \eta \nabla f(x^{(k)}) \right)$$

Exponential weights as a form of mirror descent:

Using **negative entropy** (up to an affine term): $h(x) = \sum_i (x_i \log x_i - x_i)$

Bregman divergence: $D_h(x || x^{(k)}) = h(x) - \left[h(x^{(k)}) + \nabla h(x^{(k)})^T (x - x^{(k)}) \right]$

Observe: $\frac{\partial}{\partial x_i} (x_i \log x_i - x_i) = 1 * \log x_i + x_i * \frac{1}{x_i} - 1 = \log x_i$ (product rule), note $\frac{\partial}{\partial x_i} (-x_i) = -1$

Thus, $\nabla h(x) = (\log x_1, \dots, \log x_n)$ and $\nabla h^{-1}(y) = e^{y_1}, \dots, e^{y_n}$

$$D_h(x || x^{(k)}) = \sum_i (x_i \log x_i - x_i) - [\sum_i (x_i^{(k)} \log x_i^{(k)} - x_i^{(k)}) + \sum_i \log x_i^{(k)} (x_i - x_i^{(k)})]$$

Collecting terms and rearranging, we get:

$$D_h(x || x^{(k)}) = \sum_i \left[x_i \log \left(\frac{x_i}{x_i^{(k)}} \right) - x_i + x_i^{(k)} \right]$$

If x_i and $x_i^{(k)}$ are probability vectors in a probability simplex, then $\sum_i x_i = \sum_i x_i^{(k)} = 1$.

What remains is the **Kullback-Leibler (KL)-divergence**, which measures how different one

probability distribution p is from another distribution q : $D_{KL}(p || q) = \sum_i \left[p_i \log \left(\frac{p_i}{q_i} \right) \right]$

Using distance functions in mirror descent

We can write

$$x^{(k+1)} = \operatorname{argmin}_x \{ \eta \langle \nabla f(x^{(k)}), x \rangle + D_{KL}(x || x^{(k)}) \}$$

We can also use $x^{(k+1)} = \nabla h^{-1} \left(\nabla h(x^{(k)}) - \eta \nabla f(x^{(k)}) \right)$
and $\nabla h(x) = (\log x_1, \dots, \log x_n)$ and $\nabla h^{-1}(y) = e^{y_1}, \dots, e^{y_n}$

In the dual space, the coordinate corresponding to x_i is $y_i = \log x_i$

Now, **entropic** mirror descent updates in the dual space become (componentwise)

$$x_i^{(k+1)} = e^{\log x_i^{(k)} - \eta \nabla f(x^{(k)})_i} = x_i^{(k)} e^{-\eta \nabla f(x^{(k)})_i}$$

This yields the **multiplicative weights update (MWU) algorithm**, which is also referred to as **exponential weights**.

Hedge is exponential weights for the **expert advice problem**, where the loss function is linear and the actions to choose from lie in a probability simplex.

Online (convex) optimization

From now on, we focus on **online (convex optimization) problems**.

Standard convex optimization minimizes a known, fixed, convex function $f(x)$.

Online convex optimization faces a sequence of unknown convex functions.

We're changing notation to reflect the “online” setting:

- There are T rounds.
- At each round $t = 1, \dots, T$: $x_t \in X$ is chosen first.
- Then a convex loss function $L_t(x_t): X \mapsto \mathbb{R}$ is revealed, and the learner suffers loss.

The expert-advice problem

There are n experts and T rounds.

1. At each round t : the learner picks a distribution $x_t \in \Delta^n$ over n experts.
2. Each expert i gives advice (or equivalently, suffers a loss $L_{i,t} \in [0,1]$).
3. Then the learner observes all losses and suffers expected loss

$$L_t(x_t) = \langle x_t, L_t \rangle = \sum_i x_{i,t} L_{i,t}.$$

4. The learner observes the entire loss vector L_t , i.e. he gets **full-information**.

The goal is to make the learner's cumulative loss $\sum_t \langle x_t, L_t \rangle$ decreasing wrt. that of the best fixed expert in hindsight, aka. **regret** (see below).

The Hedge algorithm

Entropic Mirror Descent

$$x_i^{(k+1)} = e^{\log x_i^{(k)} - \eta \nabla f(x^{(k)})_i} = x_i^{(k)} e^{-\eta \nabla f(x^{(k)})_i}$$

Becomes

$$x_{i,t+1} = e^{\log x_{i,t} - \eta \nabla [L_t(x_t)]_i} = x_{i,t} e^{-\eta [L_t(x_t)]_i}$$

with a **linear objective function** $\langle L_t, x \rangle$ such that $\nabla L_t(x_t)_i = L_{i,t}$:

$$x_{i,t+1} = x_{i,t} e^{-\eta L_{i,t}}.$$

For simplex-constrained problems, entropy geometry can yield better regret/convergence behavior than Euclidean geometry.

For problems **on the (probability) simplex** or with sparse/probability-like decisions, negative entropy is better adapted to that space. The exponential form keeps coordinates positive, and with normalization, we get:

$$p_{i,t} = \frac{x_{i,t}}{\sum_j x_{j,t}}$$

No Euclidean projection is needed; normalization keeps the vector on the simplex!

For linear losses on the simplex, the normalized exponential-weights update is called **Hedge**.

Hedge example

Suppose you have 2 experts and want to decide how much trust to put in each.

Start with equal weights: $x_{1,1} = 1, x_{2,1} = 1, \eta = 1$

Initial probabilities: $p_{1,1} = p_{2,1} = 1/2$

First round losses: $L_1 = (0.2, 1.0)$

Round 1 weight updates: $x_{i,t+1} = x_{i,t} e^{-\eta L_{i,t}}$

$$x_{1,2} = 1 \cdot e^{-0.2} \approx 0.819, x_{2,2} = 1 \cdot e^{-1} \approx 0.368,$$

Probabilities: $p_{i,t} = \frac{x_{i,t} e^{-\eta L_{i,t}}}{\sum_j x_{j,t} e^{-\eta L_{j,t}}}$

$$p_{1,2} = \frac{0.819}{0.819 + 0.368} \approx 0.69, p_{2,2} = \frac{0.368}{0.819 + 0.368} \approx 0.31$$

Second round losses: L_2

Round 2 weight updates: $x_{1,3} = 0.819 e^{-\eta L_{1,2}}$ and $x_{2,3} = 0.368 e^{-\eta L_{2,2}}$

etc.

Hedge pseudocode

Input:

- n : number of decisions (actions)
- T : number of rounds
- η : learning rate parameter

Hedge assumes **full-information feedback**, where the losses of all actions are observed.

Initialization:

1. Create an array of weights of length n , initializing each element to $x_{i,t}=1$.

Main loop (for t in 1 to T):

2. Normalize weights to obtain a probability distribution P over the decisions:

$$p_{it} = \frac{x_{i,t}}{\sum_j x_{j,t}}$$

3. Choose an action $I_t \sim p_t$ based on the probability distribution P .
4. Observe the **loss vector** $(L_{1t}, \dots, L_{nt})$ for all possible decisions.
5. Update the weights **of each action** i based on the loss vector L_t and learning rate η :

$$x_{i,t+1} = x_{i,t} e^{-\eta L_{i,t}}$$

End of main loop

The multi-armed bandit problem

Definition (Multi-Armed Bandit Problem): We have a set of real distributions or actions $A = \{R_1, \dots, R_n\}$, each distribution being associated with the rewards delivered by one of the n levers. The agent iteratively plays one lever per round and observes the associated reward. The objective is to maximize the sum of the collected rewards.

- Overall process:
 - The algorithm picks a mixed strategy (i.e., a probability distribution p_t over the actions (or distributions) in the model.
 - An adversary or nature pick a loss function (**adversarial or stochastic bandit model**).
 - An action is drawn from the probability distribution, and the algorithm experiences a loss for that action.
 - The algorithm **learns just the cost for the specific action chosen (aka. bandit feedback)**. This is the difference to the **expert advice problem**, where agents observe full information.

The objective is to find a balance between **exploration** (trying out different actions) and **exploitation** (choosing the best-known action).

The multi-armed bandit problem



Each machine provides a random reward from a probability distribution specific to that machine. The gambler wants to maximize the sum of rewards earned through a sequence of lever pulls.



Iteratively, for time $t = 1, 2, \dots$:

1. Nature chooses payoffs or losses on the machines L_t
2. Player picks a machine $x_t \in \Delta^n, I \sim x_t$
3. Loss is revealed for L_{It}



Process is repeated T times!
(with different, unknown, arbitrary losses every round)



Balance **exploration** (of new machines) with **exploitation**!

Simple greedy strategies:

- Exploration phase followed by a pure exploitation phase.
- The best lever is selected for a proportion $1 - \varepsilon$ of the trials, and a random lever for a proportion ε .

Feedback models

The optimizers might have different types and amounts of information available.

In the oracle model, the optimizer gains access to each loss function via a black-box feedback mechanism. An oracle $O_{L_t}(x)$ for a function $L_t: X \rightarrow \mathbb{R}$ can include:

- **Bandit feedback:** $O_{L_t}(x_t) = L_t(x_t)$
 - The oracle returns only the realized loss at the chosen action.
- **Full information feedback:** $O_{L_t} = L_t$
 - The oracle returns the entire loss function, or in a finite-action case the full loss vector.
- **Gradient feedback:** (aka. first-order feedback) $O_{L_t}(x_t) = \nabla L_t(x_t)$
 - The oracle returns the gradient at the chosen point.
- **Noisy (n-th order) feedback:**
 - $O_{L_t}(x_t) = L_t(x_t) + \varepsilon_t$ for some additive noise variable ε .
 - $O_{L_t}(x_t) = \nabla L_t(x_t) + \varepsilon_t$ for some observational noise variable ε .
aka. stochastic first-order oracle.

How are online instances generated?

Stochastic model

- There are multiple arms (or actions) to choose from, and each arm provides a stochastic reward drawn from an unknown distribution.
- Variations are:
 - **Random permutation model:** The sequence of actions or inputs is generated according to a random permutation of a fixed set of elements.
 - **Random order model:** The sequence of inputs or actions is generated randomly without any specific constraints

Adversarial model

- The key difference from the stochastic setting is that the rewards are not necessarily drawn from a fixed distribution, and an adversary can choose the rewards in a way that makes it harder for the learner to achieve good performance.
 - **Fully adversarial model:** the adversary has complete knowledge of the actions taken by the online algorithm at each step
 - **Oblivious adversarial model:** the adversary selects its sequence of actions without any knowledge of the actions taken by the online algorithm

How to evaluate online optimization algorithms?

The **competitive ratio** of an online algorithm is the worst case of the cost of the algorithm (which does not know the future) divided by the cost of the optimum offline solution.

Theorem: Against the unconstrained dynamic offline benchmark, competitive-ratio guarantees are impossible for adversarial online learning with two actions.

There are 2 arms $\{A, B\}$. The adversary will work as follows: if $p_t(A) \geq 1/2$ then the adversary sets $L_t(A) = 1$ and $L_t(B) = 0$, and otherwise $p_t(B) \geq 1/2$ and the adversary sets $L_t(B) = 1$ and $L_t(A) = 0$.

t	$\mathbf{p}_t = p_t(A), p_t(B)$	$L_t(A), L_t(B)$	Expected loss (algo)	Best hindsight action	Hindsight loss
1	(0.7,0.3)	(1,0)	0.7	B	0
2	(0.4,0.6)	(0,1)	0.6	A	0
3	(0.5,0.5)	(1,0)	0.5	B	0
4	(0.2,0.8)	(0,1)	0.8	A	0

Total expected loss of the algorithm: $0.7 + 0.6 + 0.5 + 0.8 = 2.6$

Hindsight loss: $0 + 0 + 0 + 0 = 0$

Regret minimization

Instead of average loss, we minimize **regret** in online optimization as an achievable goal:

$$Reg(T) = \sum_t L_t(x_t) - \min_{x \in X} \sum_t L_t(x)$$

where x is a (hypothetical) ex-post optimal fixed parameter vector.
 x_t is chosen by our online algorithm.

This is the difference between the aggregate loss incurred by the agent after T stages and that of the best action $x^* \in \operatorname{argmin}_{x \in X} \sum_t L_t(x)$ in hindsight.

The main goal in online optimization is to design online policies that achieve **vanishing / no regret**, i.e., for any sequence of loss functions $L_t, t = 1, 2, \dots, T$, i.e., the regret grows sublinearly in T .

$$\lim_{T \rightarrow \infty} \frac{Reg(T)}{T} = 0$$

Follow-the-Leader algorithm

- **Follow the Leader (FTL)** play the action that is optimal in hindsight up to stage t
- Minimize the sum of all losses that you encountered for all data points $s < t$ so far (greedy):

$$x_{t+1} = \operatorname{argmin}_{x \in X} \sum_{s < t} L_s(x)$$

- The policy requires a **full information oracle** and the ability to compute the arg min in the FTL update rule.
- FTL is used in **fictitious play** in game theory (an early method to compute a Nash equilibrium for certain games).
- FTL is not a no-regret learner in general (see adversarial setting on the next slide).

FTL is not a no-regret algorithm

- The player chooses $x_t \in X = [-1,1]$ while $g_t \in [-1,1]$ is *chosen by the adversary*.
 - So the static loss function is replaced by $L_t(x) = g_t x$, which might change at every t .
- The player arbitrarily picks $x_1 = 0$, and the adversary wants the player to incur losses. The adversary knows that the FTL strategy is deterministic.
- The be-the-leader (BTL) strategy cheats by looking into the next example.

t	FTL x_t	g_t	$\sum_{s=1 \dots t-1} g_s$	FTL $L_t(x)$	BTL $x'_t = x_{t+1}$	BTL $L_t(x)$
1	0	0.5	0.5	0	-1	-0.5
2	-1	-1	-0.5	1	1	-1
3	1	1	0.5	1	-1	-1
4	-1	-1	-0.5	1	1	-1

- $O(T)$ -Regret of FTL: $(T - 1) - (-0.5) \approx T$ with 0 being the best fixed strategy.
- Regret of BTL: $\approx -T$

Follow-the-Regularized-Leader

Follow the Regularized Leader (FTRL) chooses an action, which would have minimized the aggregated loss (in hindsight)

$$x_{t+1} = \operatorname{argmin}_{x \in X} \sum_{t=0}^T L_t(x) + \eta^{-1} h(x)$$

where $h: X \rightarrow \mathbb{R}$ is a regularization function and η^{-1} is the regularization weight.

$h(x)$ is continuous and strongly convex.

- For $h \equiv 0$, it is „Follow the Leader“ (FTL), and is related to Fictitious Play
- FTRL is related to policies generated by „Smooth Fictitious Play“ in game theory
- FTRL and mirror descent are closely related
- FTRL **sub-linear regret** $O(\sqrt{T})$ because the regularizer does not overreact to each new loss.

Hedge as an entropy-regularized FTRL.

Exp3 as a bandit version of entropy-FTRL/Hedge.

Recap: Hedge pseudocode

Input:

- n : number of decisions (actions)
- T : number of rounds
- η : learning rate parameter

Hedge assumes **full-information feedback**, where the losses of all actions are observed.

Initialization:

1. Create an array of weights of length n , initializing each element to $x_{i,t}=1$.

Main loop (for t in 1 to T):

2. Normalize weights to obtain a probability distribution P over the decisions:

$$p_{it} = \frac{x_{i,t}}{\sum_j x_{j,t}}$$

3. Choose an action $I_t \sim p_t$ based on the probability distribution P .
4. Observe the **loss vector** $(L_{1t}, \dots, L_{nt})$ for all possible decisions.
5. Update the weights **of each action** i based on the loss vector L_t and learning rate η :

$$x_{i,t+1} = x_{i,t} e^{-\eta L_{i,t}}$$

End of main loop

Exponential-Weights algorithm for Exploration and Exploitation (Exp3)

Input:

- n : number of decisions (actions)
- T : number of rounds
- η : learning rate parameter
- γ : exploration rate parameter

Exp3 assumes **bandit feedback**,
where the losses of one action is observed.

Initialization:

1. Create an array of weights of length n , initializing each element to $x_{i,t}=1$.

Main loop (for t in 1 to T):

2. Normalize weights to obtain a probability distribution P over the decisions:

$$p_{i,t} = (1 - \gamma) \frac{x_{i,t}}{\sum_j x_{j,t}} + \frac{\gamma}{n}$$

3. Choose an action $I_t \sim x_t$ based on the probability distribution P .

4. Observe the **loss** (L_{I_t}) .

5. **Estimate the loss for each arm i as:**

$$\hat{L}_{i,t} = L_{i,t}/p_{i,t} \text{ if } i == I_t \text{ else } 0 \quad // \text{ estimated losses}$$

6. Update the weights **of each action** i based on the loss vector L_t and learning rate η :

$$x_{i,t+1} = x_{i,t} e^{-\eta \hat{L}_{i,t}}$$

End of main loop

Exp3 example

Suppose you have 2 experts and want to decide how much trust to put in each.

Start with equal weights: $x_{1,1} = 1, x_{2,1} = 1, \eta = 1$

Initial probabilities: $p_{1,1} = p_{2,1} = 1/2$

First round losses: $L_1 = (0.2, 1.0)$

Round 1 $I_1 = 1, L_1(I_1) = 0.2, \hat{L}_{1,1} = \frac{0.2}{0.5} = 0.4, \hat{L}_{2,1} = 0$

weight updates: $x_{i,t+1} = x_{i,t} e^{-\eta \hat{L}_{it}}$

$$x_{1,2} = 1 \cdot e^{-0.4} \approx 0.670, x_{2,2} = 1 \cdot e^0 = 1,$$

Probabilities: $p_{i,t} = (1 - \gamma) \frac{x_{i,t}}{\sum_j x_{j,t}} + \frac{\gamma}{n}$ with $\gamma = 0.2$, ensures that every action has positive $p_{i,t}$

$$p_{1,2} = 0.8 \frac{0.670}{0.670+1} + 0.1 \approx 0.421, p_{2,2} = 0.8 \frac{1}{0.670+1} + 0.1 \approx 0.579$$

Second round losses L_2

etc.

Examples of no-regret algorithms

- **Full information feedback**

- Follow-the-Regularized-Leader (FTRL) with full losses
- Hedge / Exponential Weights / Multiplicative Weight Update (MWU)
 - Implemented with Hedge
 - Boosting (AdaBoost) in machine learning is also related to the MWU family of algos

- **Bandit feedback**

- EXP3
 - designed for the adversarial model
- Thompson Sampling and UCB are standard stochastic-bandit algorithms.
 - They are no-regret under stochastic assumptions, but not adversarial no-regret as EXP3.

- **Gradient feedback (aka. Online Convex Optimization)**

- Online Gradient Descent (OGD)
- Online Mirror Descent (OMD) [Nemirovski, Yudin, 1983]
- Follow-the-Regularized-Leader (FTRL) versions

So where do I need all of these methods?

Machine learning (backpropagation in neural networks, etc.)

Operations management (scheduling, production planning, cutting stock, etc.)

Logistics and transportation (vehicle routing problems, bin packing, etc.)

Airlines (flight scheduling, revenue management, etc.)

Finance (portfolio optimization, etc.)

Energy systems (optimal power flow models, etc.)

Telecommunications (packet routing, hub location, etc.)

Project management

And many more!

